



# Transformers

Rowel Atienza, Ph.D.

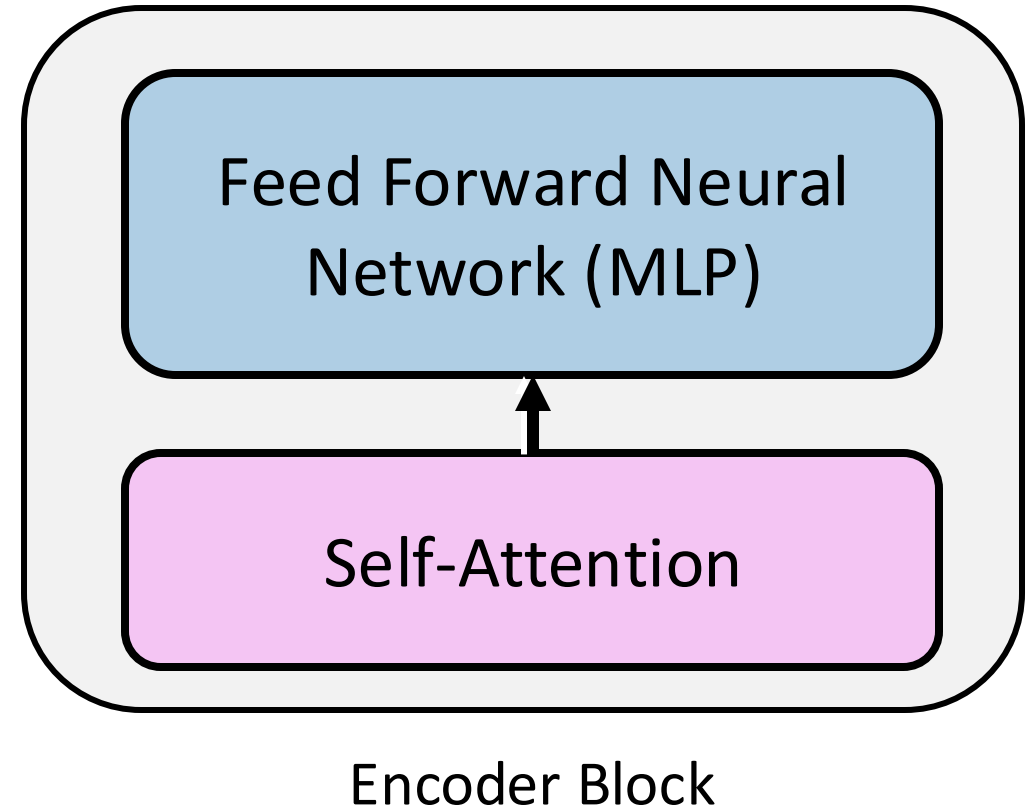
University of the Philippines

[github.com/roatienza](https://github.com/roatienza)

Updated: 2025

# Transformer Encoder/Decoder Unit Details

*Operations:* Linear, Layer Norm, Activation, Tensor Multiply/Add, Softmax



# Types of data that transformers can process

Any

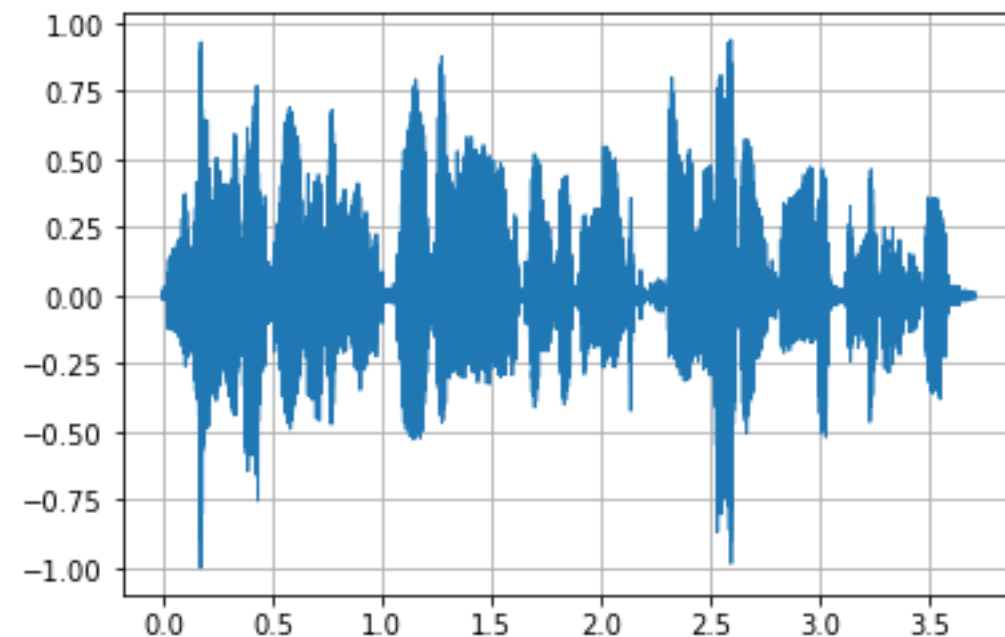


WIKIPEDIA  
The Free Encyclopedia

COCO 2017 Keypoint Detection Task



Waveform



Input Embedding is an  $n - dim$  vector

$x_1^T$

|    |    |    |    |
|----|----|----|----|
| .1 | -2 | .4 | -1 |
|----|----|----|----|

cat

$x_2^T$

|    |   |    |   |
|----|---|----|---|
| .3 | 1 | -1 | 2 |
|----|---|----|---|

opened

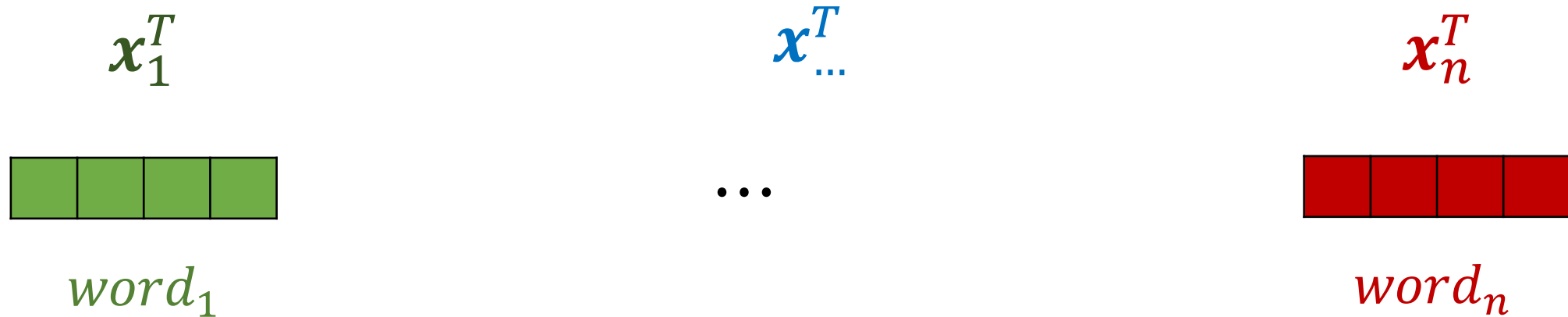
$x_3^T$

|    |    |   |    |
|----|----|---|----|
| .1 | .0 | 1 | -1 |
|----|----|---|----|

its

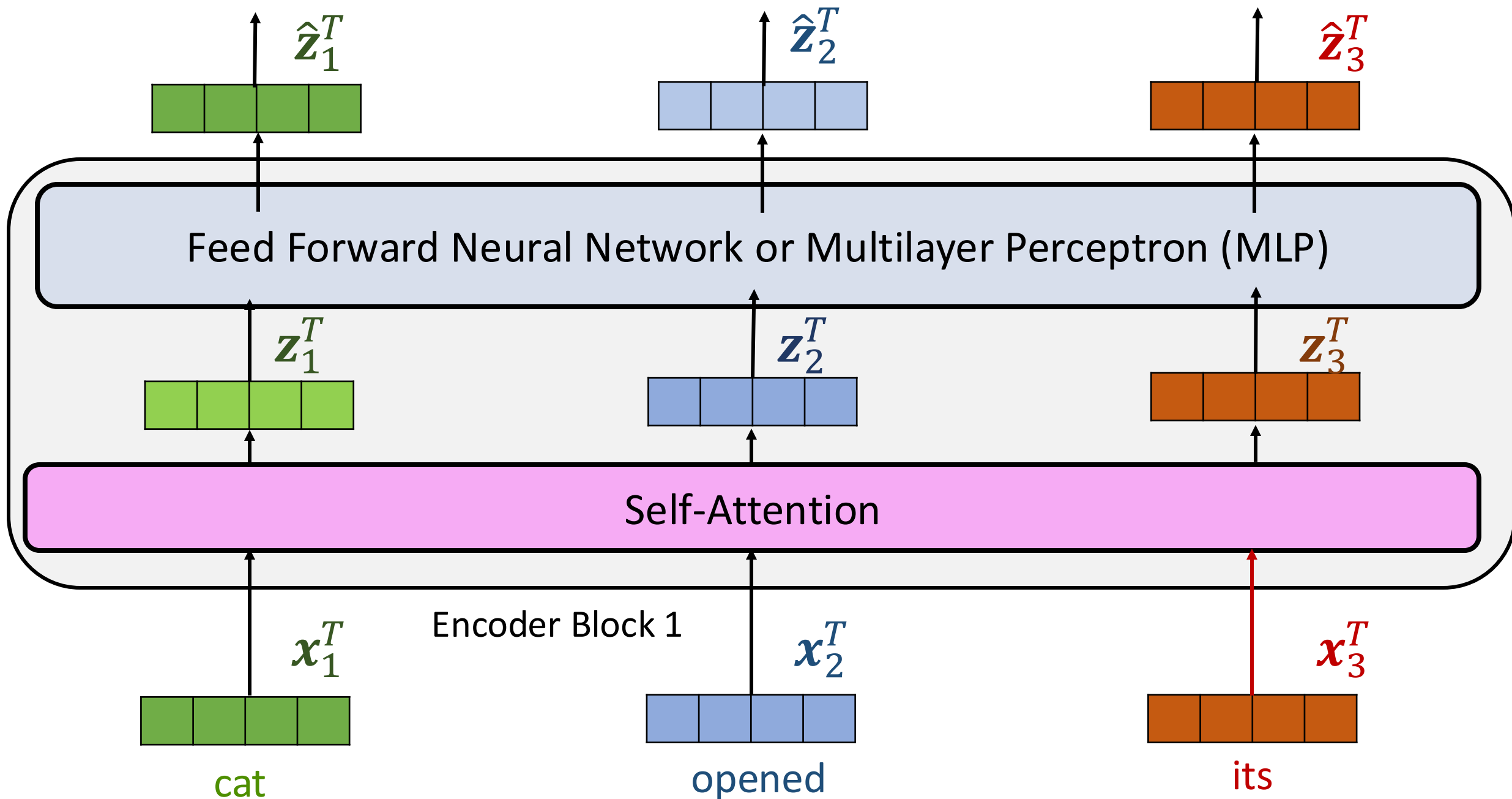
Example: Each word is converted into a 512-dim embedding vector.  
In the simple example above, it is 4-dim.

The Length of the Input is  $n$

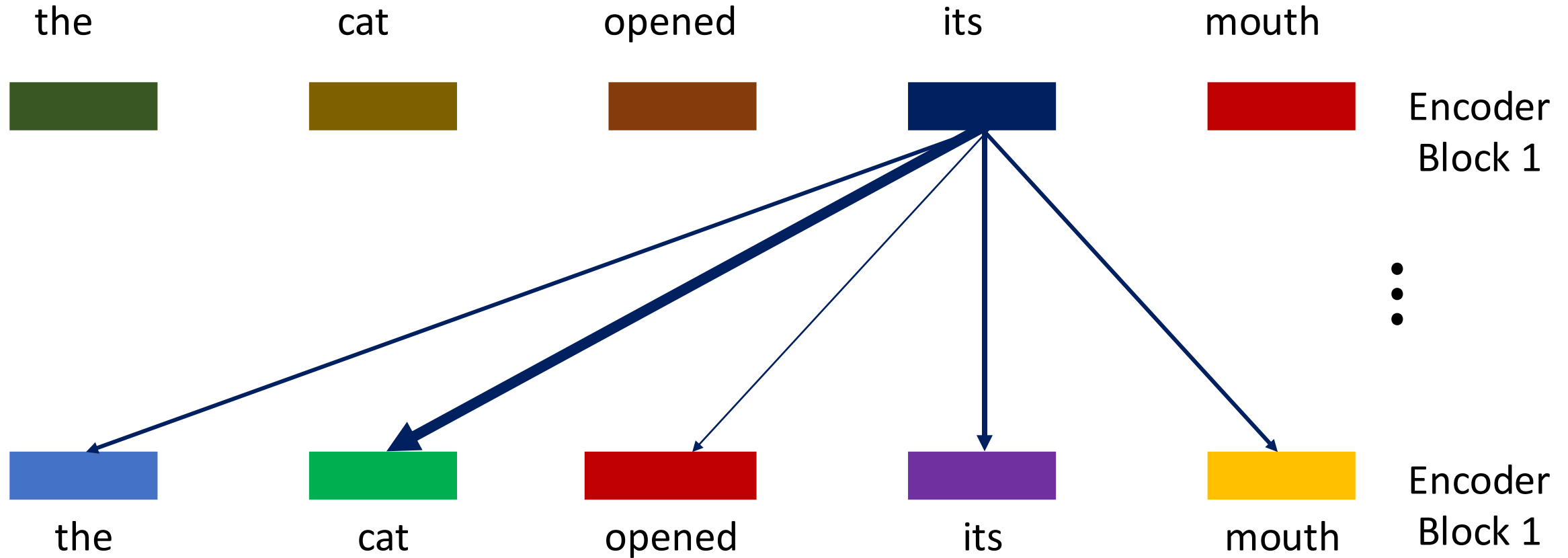


*Example:*  $n$  could be the maximum possible length of a sentence.

# Encoder with Latent Variables $\mathbf{z}_i$



# Attention between 2 words



*Attention as measured by the width of the arrow*

*Query*   *Key*   *Value*

$$\textit{Attention}(Q, K, V) = \textit{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$d_k$  is keys/queries dim (e.g. 4)

$$\textit{Attention} = \textit{softmax}\left(\frac{\begin{array}{|c|c|} \hline \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} & \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array}^T \\ \hline \sqrt{d_k} \end{array}\right) \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array}$$

$$\textit{Attention}(Q, K, V) = Z = \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array}$$

*Values* With all things considered, this is where the attention should be

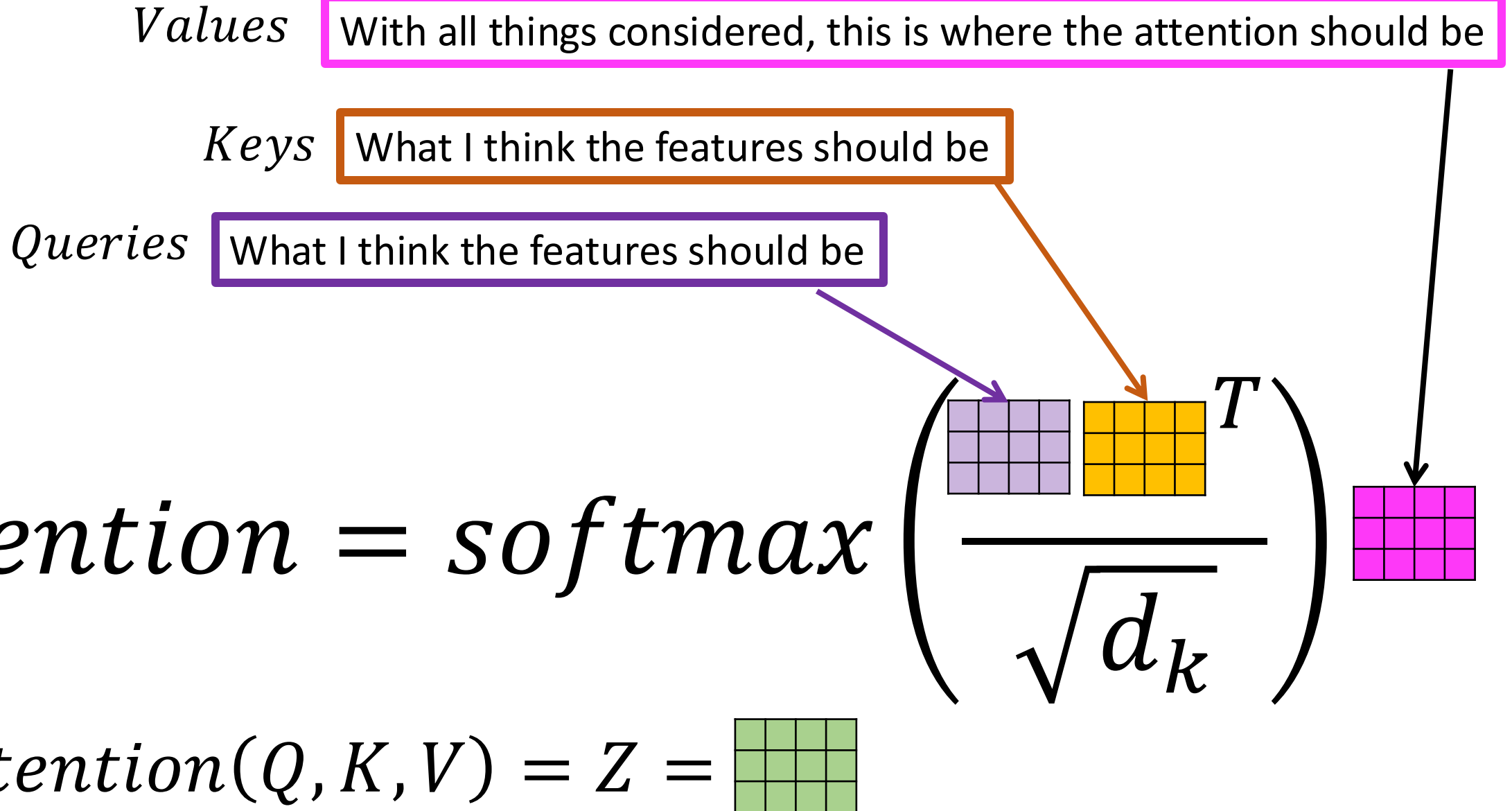
*Keys* What I think the features should be

*Queries* What I think the features should be

$$\text{Attention} = \text{softmax} \left( \frac{\begin{matrix} \text{purple grid} & \text{orange grid} \end{matrix}^T}{\sqrt{d_k}} \right)$$

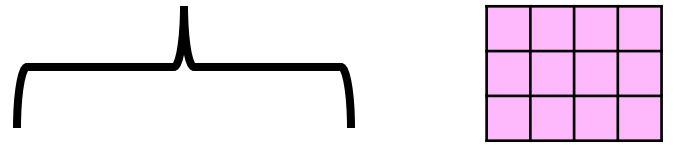
$\text{Attention}(Q, K, V) = Z = \text{green grid}$

$\text{green grid} \rightarrow \text{pink grid}$

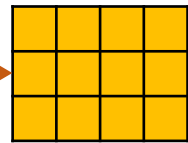


# Consider an Attention Layer Examining a Digit

I can see everything that you can see. You are a part of digit 3.

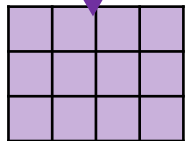


*Values*



*Keys*

I saw a bar to the left. I think I am a part of digit 3, 5, 6, or 8.



*Queries*

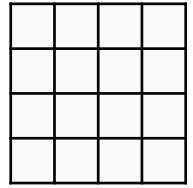
I think I am a part of digit 3. 5 is also possible.

*Example: Let us focus on the lower-right patch only*

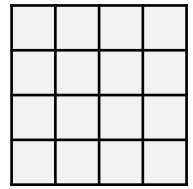
Self-Attention

Attention Layer 1 Learnable Parameters

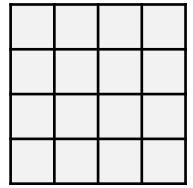
$W^Q$



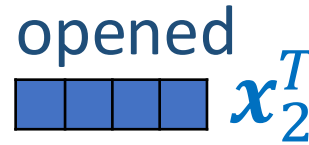
$W^K$



$W^V$



Embedding



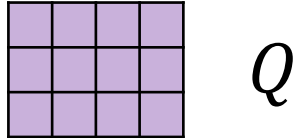
$X = \begin{bmatrix} x_1^T \\ x_2^T \\ x_3^T \end{bmatrix}$  Encoder 1 Inputs

$q_1^T = x_1^T W^Q$

$q_2^T = x_2^T W^Q$

$q_3^T = x_3^T W^Q$

Queries



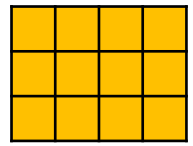
$Q = XW^Q$

$k_1^T = x_1^T W^K$

$k_2^T = x_2^T W^K$

$k_3^T = x_3^T W^K$

Keys



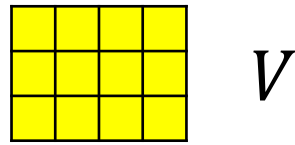
$K = XW^K$

$v_1^T = x_1^T W^V$

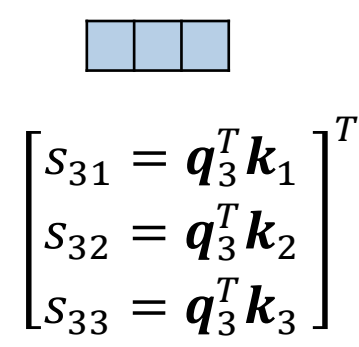
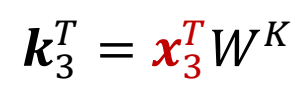
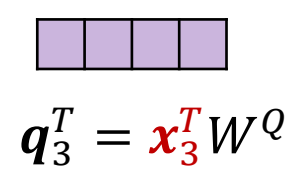
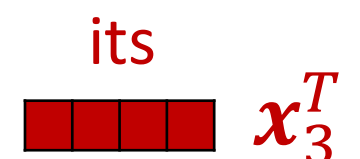
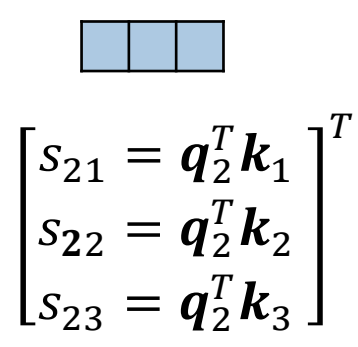
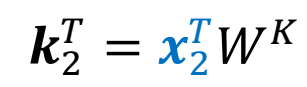
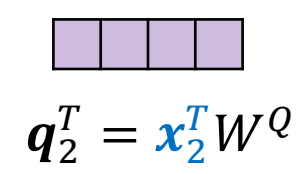
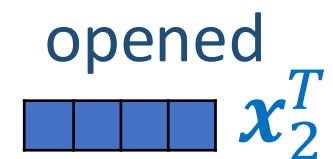
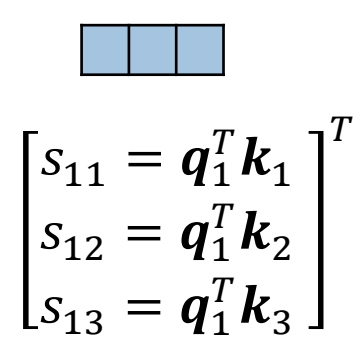
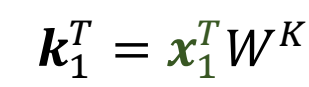
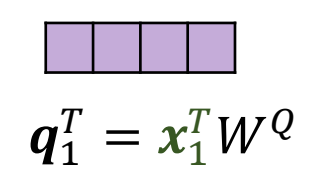
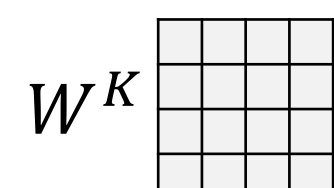
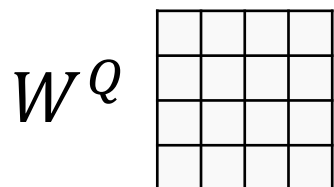
$v_2^T = x_2^T W^V$

$v_3^T = x_3^T W^V$

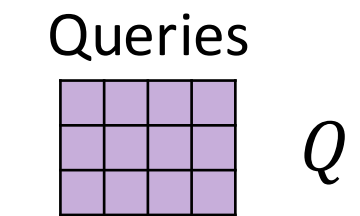
Values



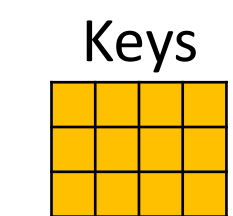
$V = XW^V$



$X = \begin{bmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \mathbf{x}_3^T \end{bmatrix}$  Encoder 1 Inputs



$Q = XW^Q$



$K = XW^K$

Scores

$\begin{pmatrix} \text{Queries} \end{pmatrix} \begin{pmatrix} \text{Keys} \end{pmatrix}^T$

$= \begin{bmatrix} \text{Scores} \end{bmatrix} = S$

Multi-Head  
(eg 8-head)

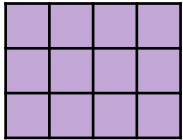
cat  $x_1^T$

opened  $x_2^T$

its  $x_3^T$

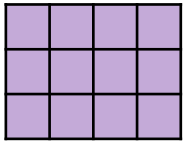
$X = \begin{bmatrix} x_1^T \\ x_2^T \\ x_3^T \end{bmatrix}$  Encoder 1  
Inputs

Queries



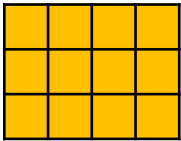
$$Q_1 = XW_1^Q$$

Queries



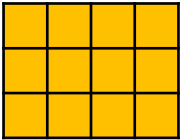
$$Q_8 = XW_8^Q$$

Keys



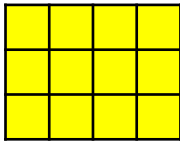
$$K_1 = XW_1^K$$

Keys



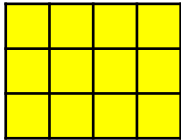
$$K_8 = XW_8^K$$

Values



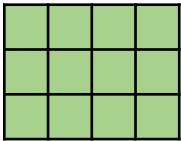
$$V_1 = XW_1^V$$

Values

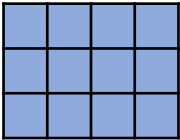


$$V_8 = XW_8^V$$

Head 1:  $Z_1$

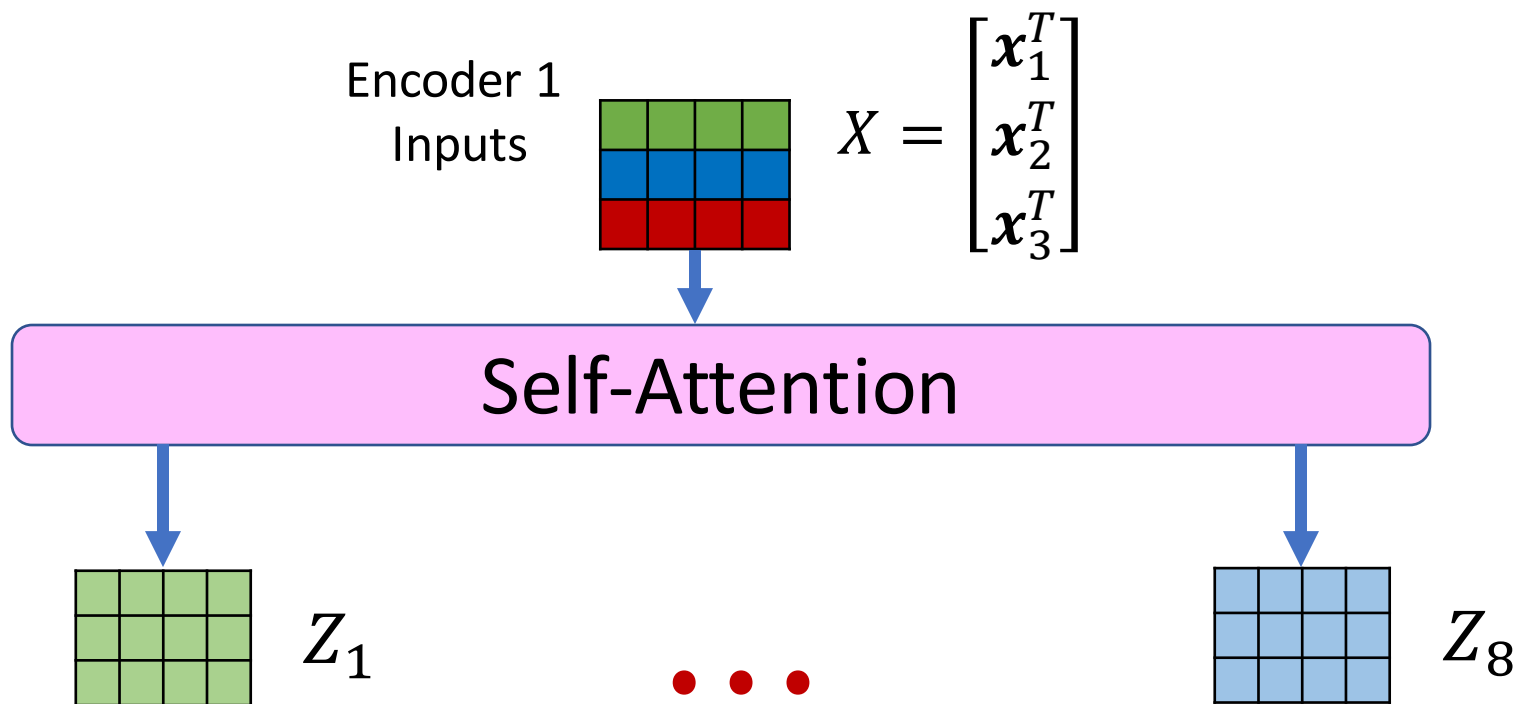


Head 8:  $Z_8$



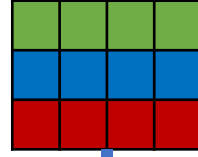
...

Multi-Head  
(eg 8-head)



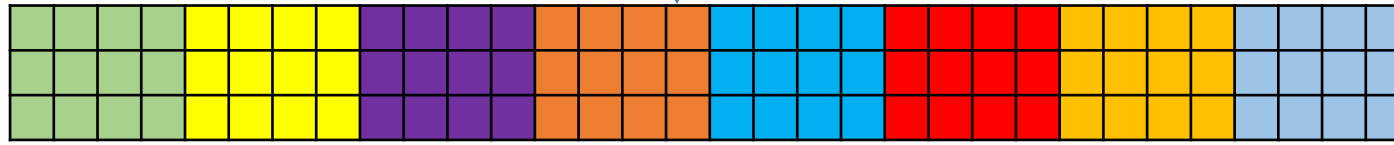
Multi-Head  
(eg 8-head)  
Merge Outputs  
Apply Weights

Encoder 1  
Inputs



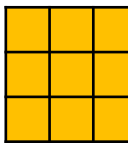
$$X = \begin{bmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \mathbf{x}_3^T \end{bmatrix}$$

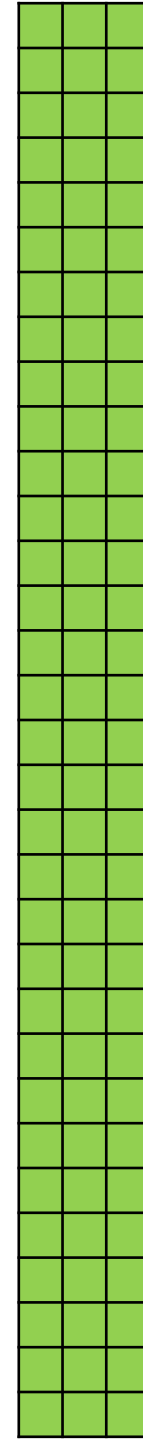
Self-Attention



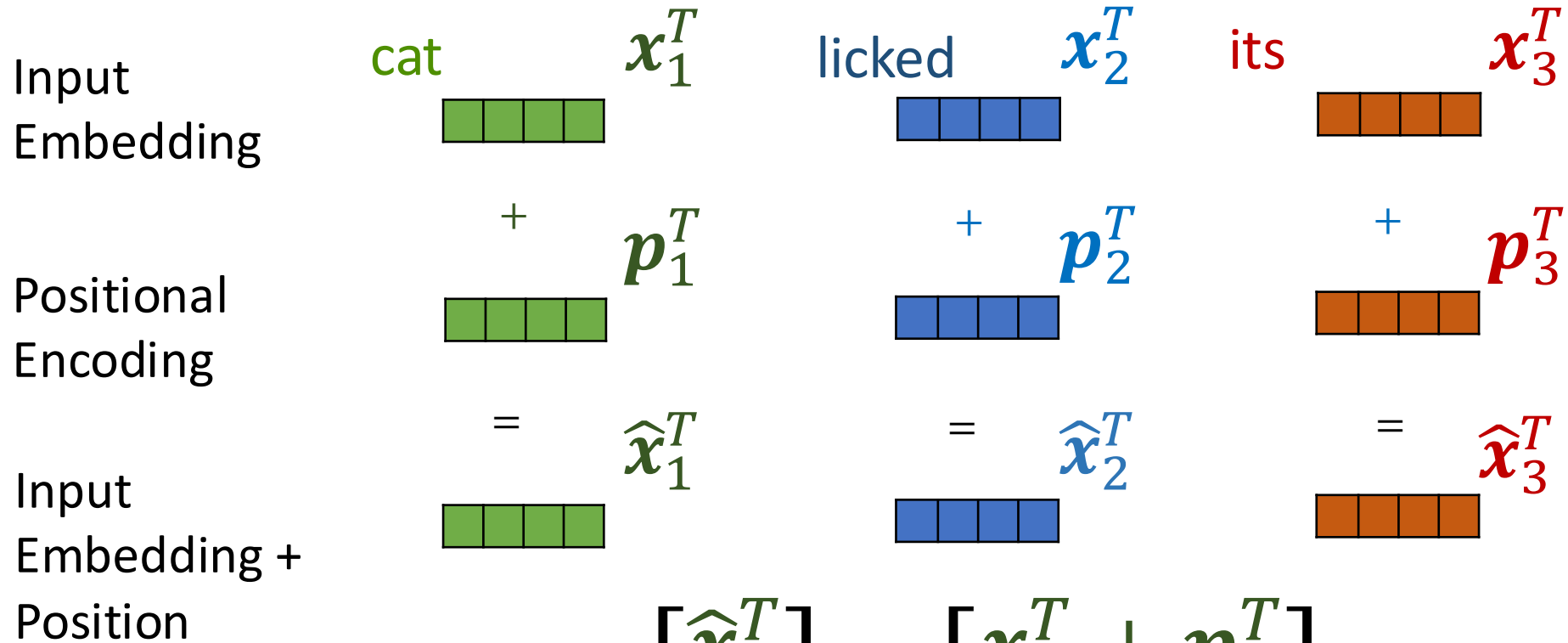
$$\text{cat}(Z_1, \dots, Z_8)$$

$$\times W^O$$

$$= Z$$
A 3x3 grid of yellow squares.



# Adding Position Info to Inputs



$$\hat{X} = \begin{bmatrix} \hat{x}_1^T \\ \hat{x}_2^T \\ \hat{x}_3^T \end{bmatrix} = \begin{bmatrix} x_1^T + p_1^T \\ x_2^T + p_2^T \\ x_3^T + p_3^T \end{bmatrix}$$

# Positional Encoding

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_k}}}\right) \quad dim = 2i \text{ is even}$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_k}}}\right) \quad dim = 2i + 1 \text{ is odd}$$

$$pos = 0, 1, \dots, n_{pos}-1$$

$$dim = 0, 1, \dots, n_{dim}-1$$

$$n = 10000, d_k \text{ is embedding dim}$$

Other positional encoding methods: learnable, rotary positional embedding

# Positional Encoding

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_k}}}\right) \quad dim = 2i \text{ is even}$$

$$d_k = 4$$

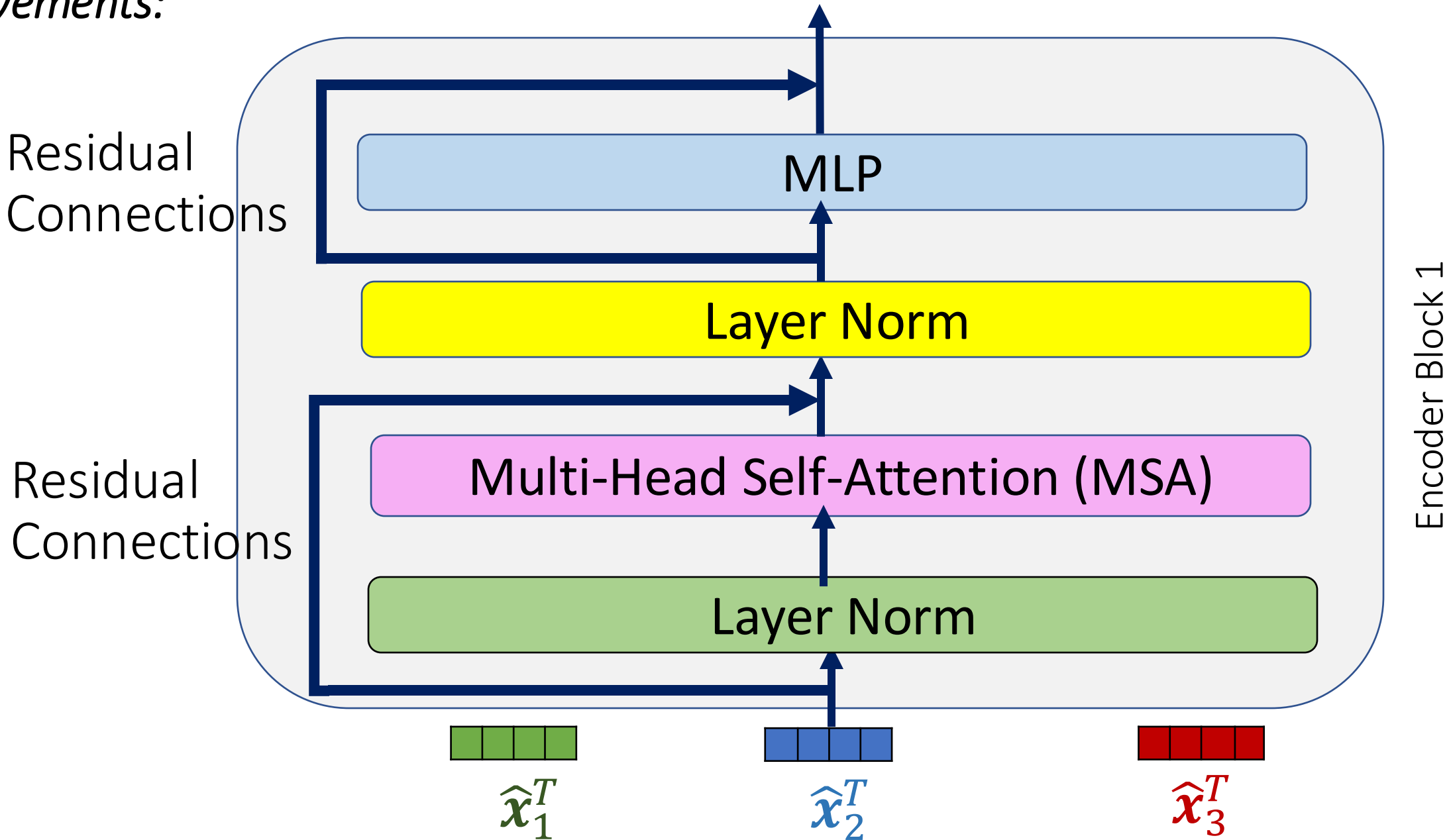
$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_k}}}\right) \quad dim = 2i + 1 \text{ is odd}$$

|        | <div><div><i>i</i></div><div><i>pos</i></div></div> | 0                                                     | 0                                                     | 1                                                     | 1                                                     |
|--------|-----------------------------------------------------|-------------------------------------------------------|-------------------------------------------------------|-------------------------------------------------------|-------------------------------------------------------|
| cat    | 0                                                   | $\sin\left(\frac{0}{10000^{\frac{2(0)}{d_k}}}\right)$ | $\cos\left(\frac{0}{10000^{\frac{2(0)}{d_k}}}\right)$ | $\sin\left(\frac{0}{10000^{\frac{2(1)}{d_k}}}\right)$ | $\cos\left(\frac{0}{10000^{\frac{2(1)}{d_k}}}\right)$ |
| opened | 1                                                   | $\sin\left(\frac{1}{10000^{\frac{2(0)}{d_k}}}\right)$ | $\cos\left(\frac{1}{10000^{\frac{2(0)}{d_k}}}\right)$ | $\sin\left(\frac{1}{10000^{\frac{2(1)}{d_k}}}\right)$ | $\cos\left(\frac{1}{10000^{\frac{2(1)}{d_k}}}\right)$ |
| its    | 2                                                   | $\sin\left(\frac{2}{10000^{\frac{2(0)}{d_k}}}\right)$ | $\cos\left(\frac{2}{10000^{\frac{2(0)}{d_k}}}\right)$ | $\sin\left(\frac{2}{10000^{\frac{2(1)}{d_k}}}\right)$ | $\cos\left(\frac{2}{10000^{\frac{2(1)}{d_k}}}\right)$ |

| Sequence |   | Index<br>of token,<br>$k$ | Positional Encoding<br>Matrix with $d=4$ , $n=100$ |                              |                               |                               |                               |
|----------|---|---------------------------|----------------------------------------------------|------------------------------|-------------------------------|-------------------------------|-------------------------------|
|          |   |                           | $i=0$                                              | $i=0$                        | $i=1$                         | $i=1$                         |                               |
| I        | → | 0                         | →                                                  | $P_{00}=\sin(0)$<br>= 0      | $P_{01}=\cos(0)$<br>= 1       | $P_{02}=\sin(0)$<br>= 0       | $P_{03}=\cos(0)$<br>= 1       |
| am       | → | 1                         | →                                                  | $P_{10}=\sin(1/1)$<br>= 0.84 | $P_{11}=\cos(1/1)$<br>= 0.54  | $P_{12}=\sin(1/10)$<br>= 0.10 | $P_{13}=\cos(1/10)$<br>= 1.0  |
| a        | → | 2                         | →                                                  | $P_{20}=\sin(2/1)$<br>= 0.91 | $P_{21}=\cos(2/1)$<br>= -0.42 | $P_{22}=\sin(2/10)$<br>= 0.20 | $P_{23}=\cos(2/10)$<br>= 0.98 |
| Robot    | → | 3                         | →                                                  | $P_{30}=\sin(3/1)$<br>= 0.14 | $P_{31}=\cos(3/1)$<br>= -0.99 | $P_{32}=\sin(3/10)$<br>= 0.30 | $P_{33}=\cos(3/10)$<br>= 0.96 |

Positional Encoding Matrix for the sequence 'I am a robot'

Improvements:



# Layer Normalization vs Batch Normalization

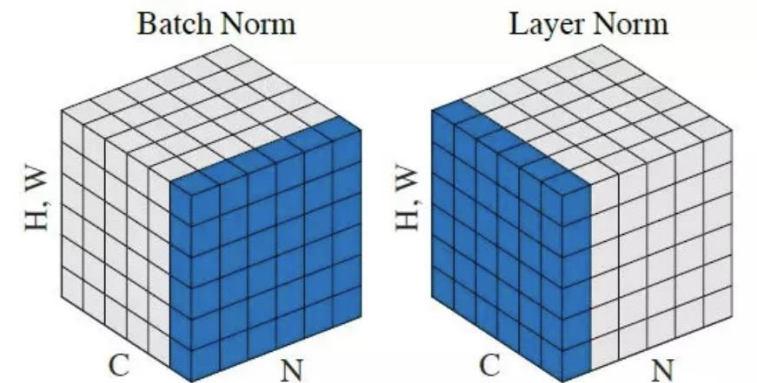
$$\mu = \frac{1}{d} \sum_i^d x_i$$

$$\sigma^2 = \frac{1}{d} \sum_i^d (x_i - \mu)^2$$

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

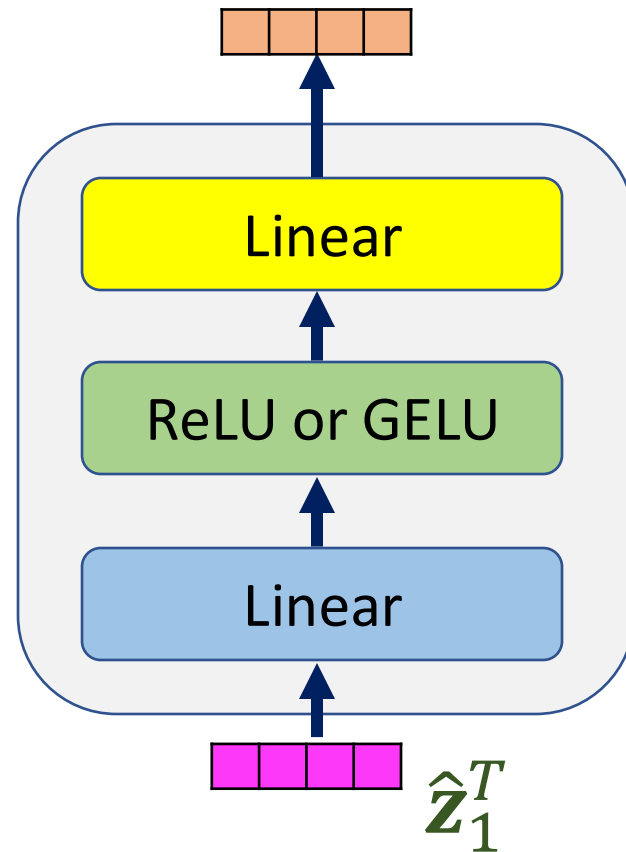
- Batch Normalization –  $d$  is across the entire batch
- Layer Normalization –  $d$  is across the layer

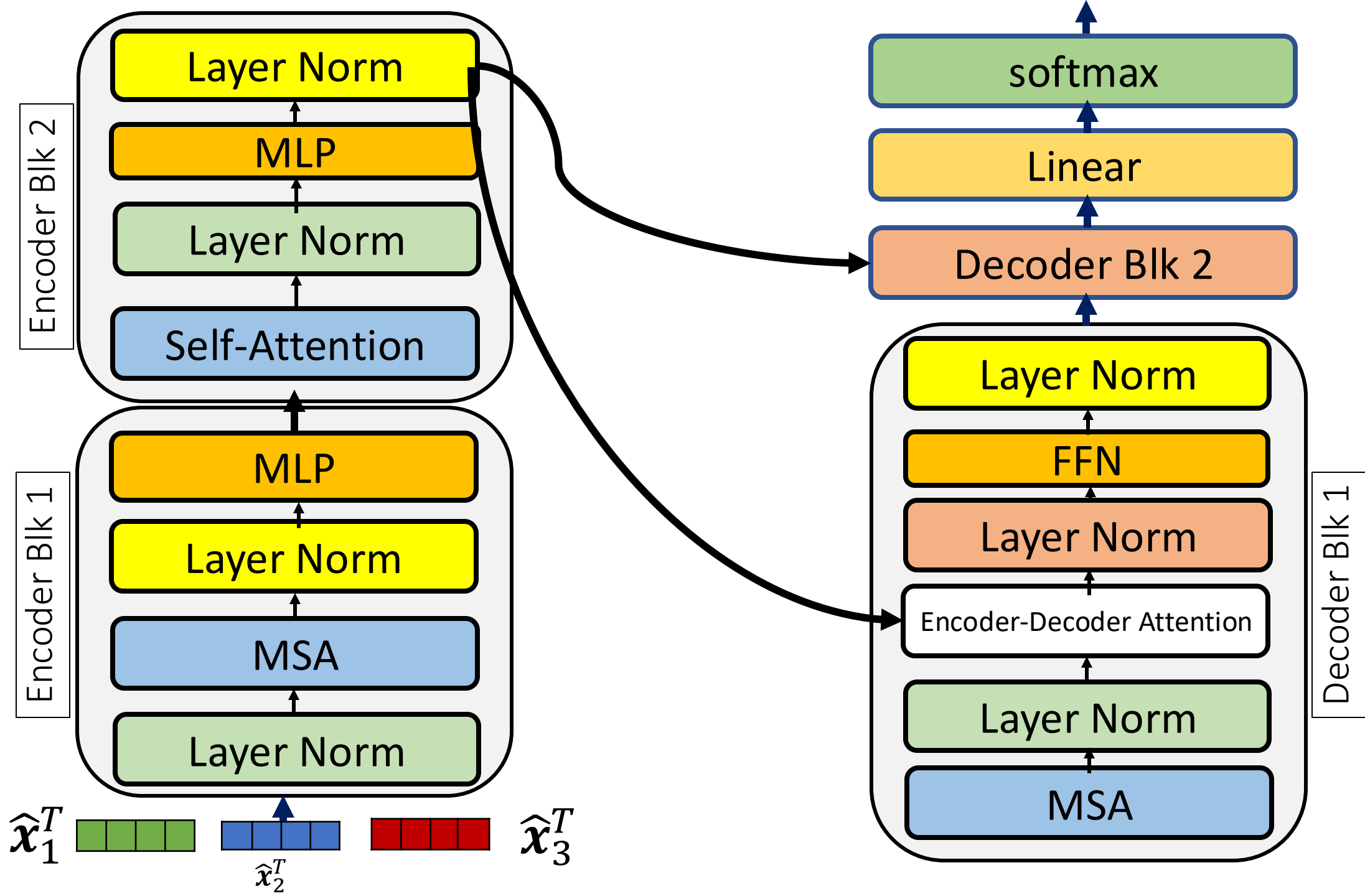
$\hat{x}_i$  is the normalized feature

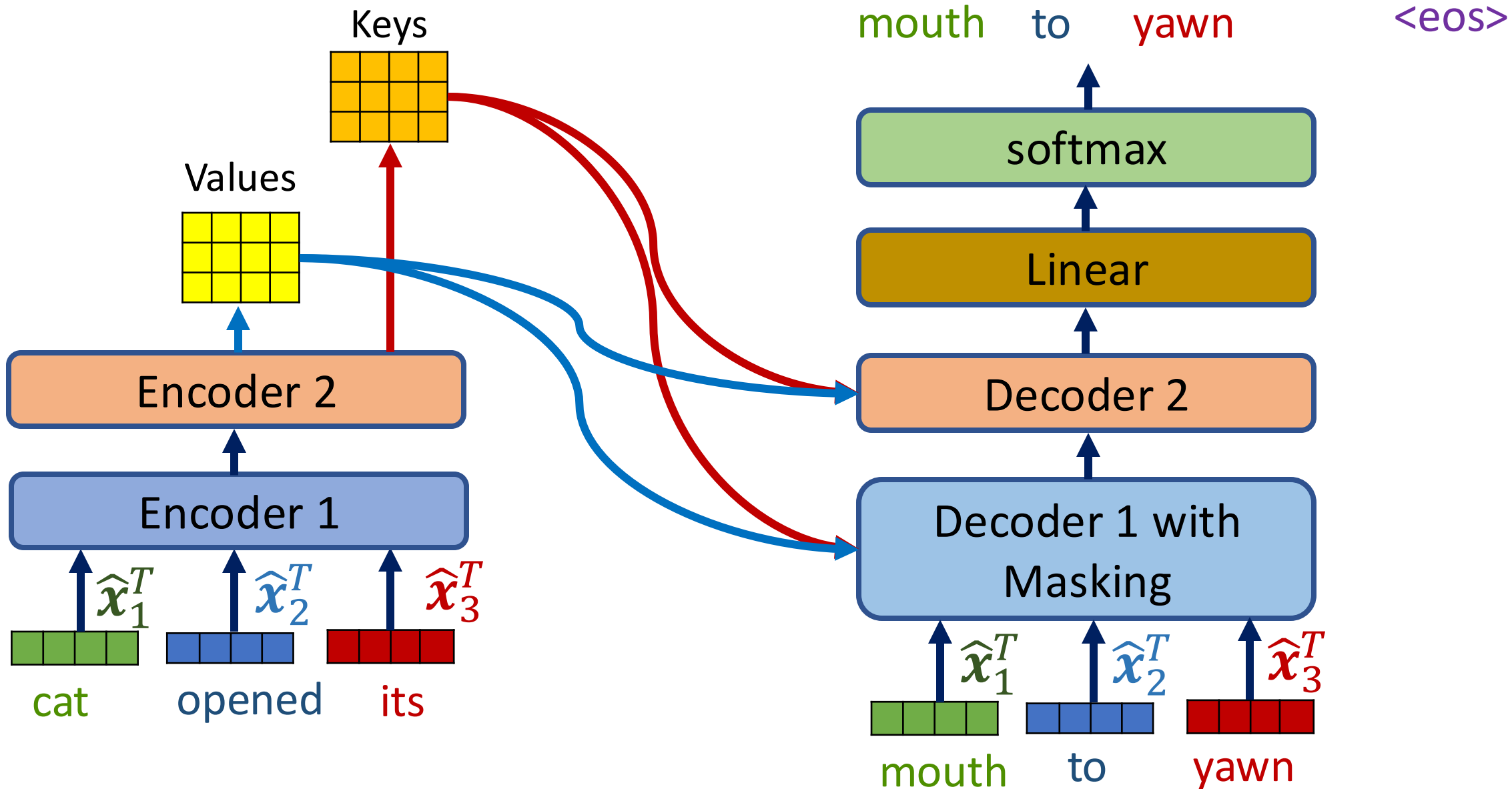


# FFN: Feed Forward Neural Network (MLP)

$$MLP(x) = \max(0, xW_1 + b_1)W_2 + b_2$$







*Masking prevents Decoder 1 from seeing the future. Decoder 1 relies only on the previous outputs.*

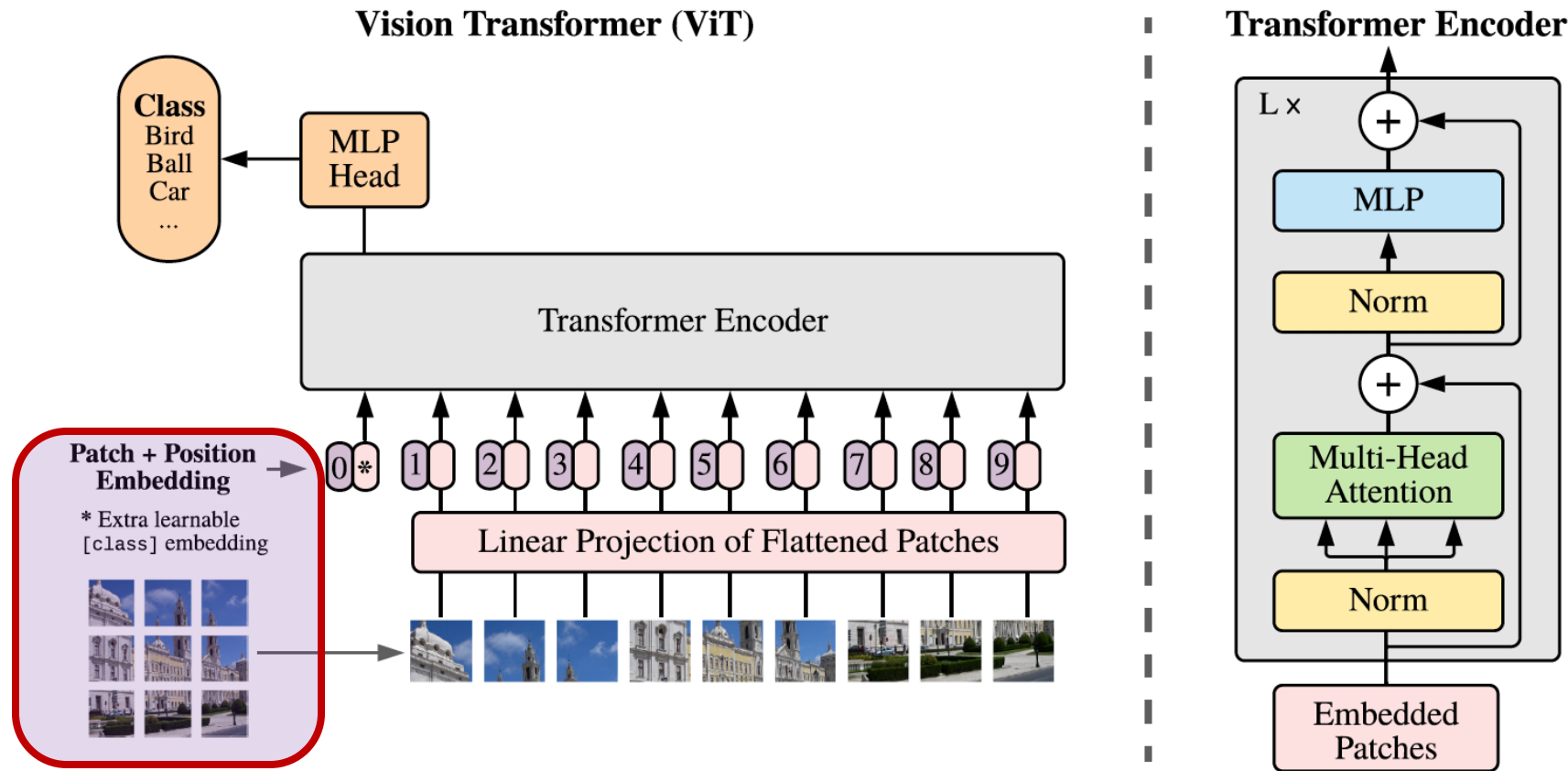


Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings to the resulting sequence of vectors, and feed the patches to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable "classification token" to the sequence. The illustration of the Transformer encoder was inspired by Vaswani et al. (2017).

AN IMAGE IS WORTH 16X16 WORDS:  
TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE, ICLR 2021

# Inductive Bias

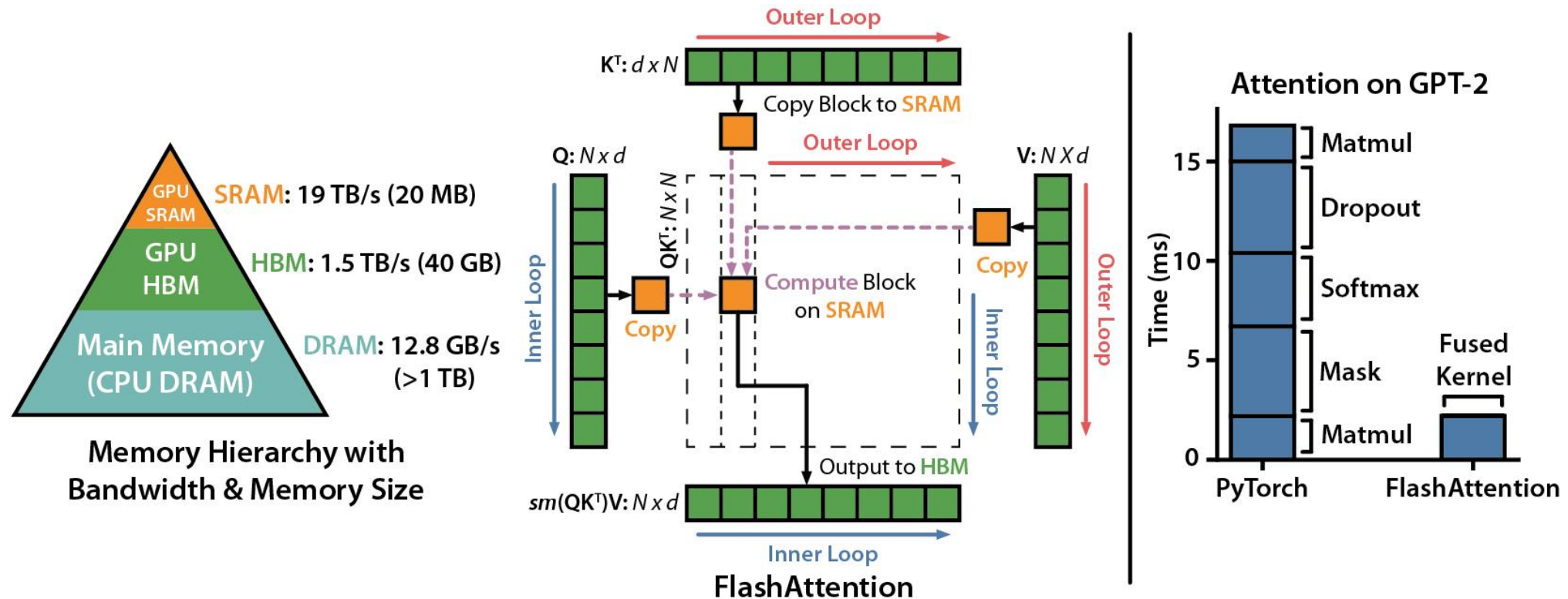
Transformers lack some inductive biases inherent to CNNs, such as translation equivariance and scale invariance (w/ maxpool), and therefore do not generalize well when trained on insufficient amounts of data.

However, the picture changes if we train the models on large datasets (14M-300M images). We find that large scale training trumps inductive bias.

# Improvements on Transformers

- Flash Attention
- Rotary Positional Embedding

# Flash Attention



Dao, Tri, et al. "Flashattention: Fast and memory-efficient exact attention with io-awareness NeuRIPS 2022

Dao, Tri. "Flashattention-2: Faster attention with better parallelism and work partitioning ICLR 2023

# Rotary Position Encoding (RoPE)

$$Q = [q_1 \quad q_2]$$

$$\rightarrow Q = [q_1 \cos p\theta - q_2 \sin p\theta \quad q_1 \sin p\theta + q_2 \cos p\theta]$$

$$K = [k_1 \quad k_2]$$

$$\rightarrow K = [k_1 \cos p'\theta - k_2 \sin p'\theta \quad k_1 \sin p'\theta + k_2 \cos p'\theta]$$

$$QK^T = [q_1 k_1 + q_2 k_2]$$

$$\rightarrow QK^T = [x_1 \cos(p + p')\theta + x_2 \cos(p - p')\theta + x_3 \sin(p + p')\theta + x_4 \sin(p - p')\theta + \dots]$$

Natural encoding of relative position within Q and K

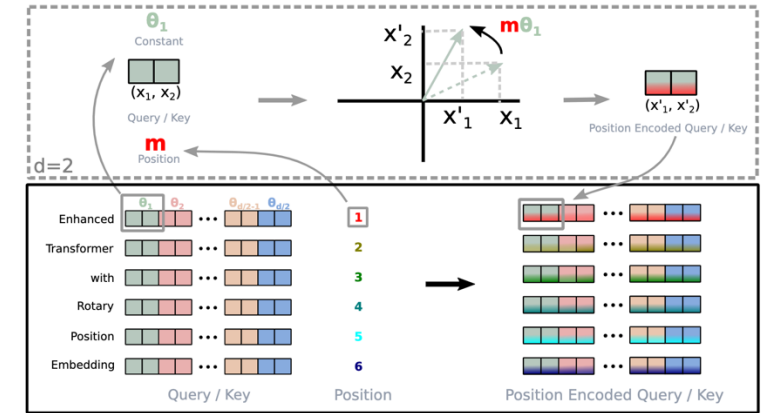


Figure 1: Implementation of Rotary Position Embedding(RoPE).

# References

Vaswani, Ashish, et al. "Attention is all you need." *Advances in neural information processing systems*. 2017.

Dosovitskiy, Alexey, et al. "An image is worth 16x16 words: Transformers for image recognition at scale." *arXiv preprint arXiv:2010.11929* (2020). ICLR 2021.

Illustrated Transformer, <http://jalammar.github.io/illustrated-transformer/>

Transformers from Scratch, <http://peterbloem.nl/blog/transformers>

Transformer Family, <https://lilianweng.github.io/lil-log/2020/04/07/the-transformer-family.html>

# In Summary

Transformers could be the most important breakthrough in the recent history of deep learning

Transformers have been used to produce state-of-the-art performances in language, vision, audio, and multi-modal domains

Expect more development in this field in the near future

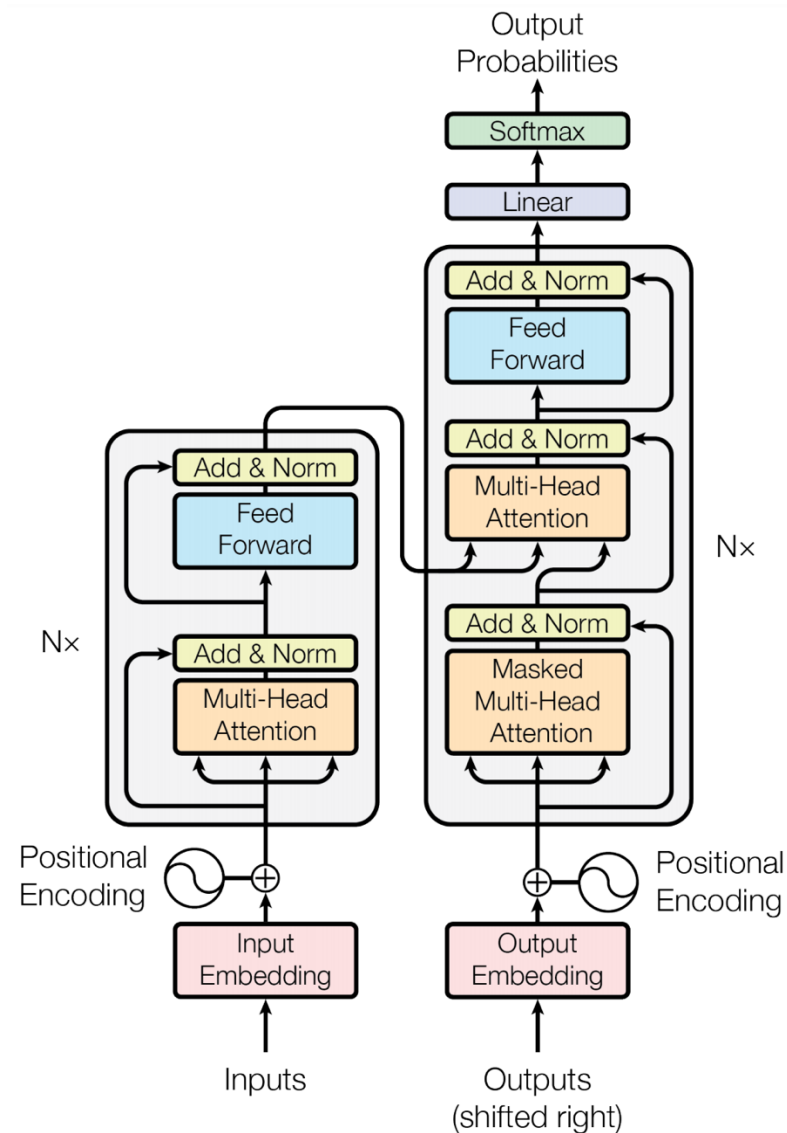


Figure 1: The Transformer - model architecture.

Vaswani, Ashish, et al. "Attention is all you need." *Advances in neural information processing systems*. 2017.

Code demo is next